

Capability-Routed Guard: Defending Large Reasoning Models Against Reasoning-Centric Jailbreaks

Yiyong Liu¹, Yixin Wu¹, Jun Sakuma²

¹CISPA Helmholtz Center for Information Security

²Institute of Science Tokyo

Abstract

Large reasoning models (LRMs) expose a new safety failure mode: adversarial prompts can manipulate reasoning context, task decomposition, or capability interpretation so that harmful objectives are processed as legitimate reasoning steps. Existing safeguards, including safety reminders, external classifiers, and self-checking wrappers, are often brittle because they either inspect the adversarial prompt directly or ask the target model to perform additional safety reasoning on the same surface that attacks exploit. We introduce *Capability-Routed Guard* (CRG), a model-agnostic inference-time guardrail for closed-source LRMs, where defenders cannot inspect hidden reasoning traces or modify model weights. CRG reframes prompt defense as a capability-routing problem: a side-channel controller first constructs a trusted representation of the user’s authorized task, active context, safety evidence, and capability-transfer risk, separating executable intent from untrusted reasoning context. This representation supports route-specific execution, allowing CRG to block high-risk requests, constrain ambiguous ones, and forward low-risk requests through trusted active context. Finally, CRG applies TraceCheck to verify consistency with the authorized task and invokes a restricted fallback to preserve utility for low-risk benign prompts. Extensive experiments demonstrate that CRG effectively mitigates diverse reasoning-centric jailbreaks while preserving benign utility and avoiding common over-refusal issues. Further analysis shows that its components contribute complementary benefits, highlighting the importance of coordinated defense mechanisms for securing large reasoning models.

Introduction

Large reasoning models (LRMs) (Wei et al. 2022; OpenAI 2024; DeepSeek-AI 2025) are increasingly used for tasks that require long-horizon planning, code synthesis, mathematical reasoning, scientific analysis, and autonomous decision support. Their strength comes from an expanded ability to decompose problems, maintain intermediate context, and reason over multi-step trajectories. This same capability, however, creates a new safety challenge. Unlike conventional chat models, where safety failures often arise from direct refusal bypasses, LRMs can be attacked through the reasoning process itself: adversarial prompts can manipulate task decomposition, inject false authority, dilute capability-transfer signals, or cause harmful objectives to appear as benign intermediate subtasks.

Recent jailbreaks (Zhao et al. 2025; Yao et al. 2025; Liang et al. 2025; Lu et al. 2025; Nguyen et al. 2026) show that this vulnerability is not limited to a single prompt pattern. Rather than merely asking the model to ignore its safety policy, these attacks increasingly shape the intermediate reasoning path: they pad harmful goals with benign-looking reasoning, guide the model through locally safe transformations, search automatically for effective reasoning wrappers, or embed unsafe objectives in elaborate fictional or encoded contexts. These strategies exploit the model’s tendency to maintain coherence with the reasoning trajectory it has accepted, even when the final answer would transfer harmful capability. CoT-Hijacking (Zhao et al. 2025) and SEAL (Nguyen et al. 2026) are examples of this broader shift, but the underlying issue is more general than any individual attack template. We refer to attacks that exploit the reasoning trajectory, authority boundary, or capability interpretation of LRMs as *reasoning-centric jailbreaks*.

Reasoning-centric jailbreaks expose limitations in many existing defense paradigms (Inan et al. 2023; Cao et al. 2025; Zhang et al. 2024; Li et al. 2025; Shen et al. 2025; Wang et al. 2026). Most safeguards are designed to detect or suppress harmful prompts and responses, rather than to govern how a reasoning model interprets, routes, and executes a request. As a result, they often lack an explicit mechanism for separating trusted task intent from untrusted reasoning context, and they do not directly regulate when the target model’s reasoning capability should be exposed, constrained, or withheld. This mismatch leads to a persistent safety–utility tension: defenses that block aggressively can over-refuse benign but safety-salient prompts, while defenses that preserve utility may still allow adversarial reasoning context to shape the target model’s behavior.

This tension is amplified for closed-source LRMs, where defenders cannot inspect hidden reasoning traces, modify model weights, fine-tune safety behavior, or change provider-side moderation policies. A practical defense must therefore operate at inference time under limited observability, reducing harmful capability transfer without sacrificing the utility that makes LRMs valuable. This constraint motivates a defense that can reason over the trustworthiness of the prompt context, decide how much target reasoning capability should be exposed, and verify public outputs without relying on model-internal access.

We propose *Capability-Routed Guard* (CRG), an inference-time guardrail for closed-source LLMs. The key idea is to shift defense from judging a prompt in isolation to governing the entire target-model execution path. CRG first constructs a trusted side-channel representation that separates the user’s authorized task and safety-relevant evidence from untrusted reasoning context. This representation is used to decide not only whether a request should be allowed, but also what context should reach the target model, how much target reasoning should be exposed, and how the final response should be verified. In practice, CRG blocks high-risk requests, routes ambiguous requests through constrained reasoning, forwards low-risk requests through trusted active context, audits generated outputs with TraceCheck, and uses a restricted fallback to preserve utility for low-risk benign prompts. This unified control over intent, context, reasoning, and verification is what distinguishes CRG from defenses that only filter or rewrite prompts.

We evaluate CRG against five reasoning-centric jailbreak families across harmful-behavior benchmarks and multiple target LLMs, while also measuring benign false positives on safety-salient and general utility datasets. CRG substantially reduces attack success across attack styles while maintaining low false-positive rates. On Gemini 2.5 Pro HarmBench, for example, CRG reduces CoT-Hijacking ASR from 100% to 12% and FicDetail ASR from 97% to 0%. Further analysis shows that CRG is not merely a stronger prompt filter: its components provide complementary safety and utility benefits across routing, context control, verification, and fallback.

Our contributions are:

- We formulate reasoning-centric jailbreaks as attacks that manipulate reasoning context, task decomposition, authority, or capability interpretation in LLMs, with particular emphasis on the closed-source deployment setting.
- We introduce CRG, a model-agnostic inference-time guardrail that uses structured side-channel representation and capability-aware routing to defend closed-source LLMs.
- We evaluate CRG against five attack families and multiple target LLMs, showing strong reductions in attack success while preserving benign utility.
- We provide component analysis, extractor sensitivity, latency analysis, and adaptive attack evaluation to study why CRG works and where inference-time LLM defense remains limited.

Related Work

Jailbreak attacks. Early jailbreak attacks mainly targeted instruction following and refusal behavior. Some attacks rely on semantic disguise, such as role play (Li et al. 2023), paraphrase (Ding et al. 2024), or encoded instructions (Jin et al. 2024). Others search for adversarial prompts automatically, for example through suffix optimization (Zou et al. 2023) or iterative red teaming (Chao et al. 2025). These lines of work show that jailbreaks are not only handcrafted prompts, but can also be systematically generated. Recent attacks broaden the surface further by exploiting long context, multi-turn interaction, indirect instructions, and fictional or transformed tasks.

For LLMs, the attack surface shifts from the final refusal boundary to the reasoning process itself. CoT-Hijacking (Zhao et al. 2025) hides harmful goals inside reasoning scaffolds. Mousetrap (Yao et al. 2025), SEAL (Nguyen et al. 2026), and AutoRAN (Liang et al. 2025) steer or search over reasoning trajectories, while FicDetail (Lu et al. 2025) uses rich fictional detail to make unsafe continuations appear contextually natural. These attacks differ in form, but they share a common effect: adversarial context changes how the target model reasons about intent, authority, and harmful capability. Our work focuses on this reasoning-centric attack surface.

Inference-time defenses and safeguards. Existing safeguards differ in where they place the safety boundary. Training-time alignment and fine-tuning (Wang et al. 2024) change the model itself, but they are not available when the target is a closed-source API model. Prompt hardening and external moderation (Zhang et al. 2024) are easier to deploy, yet they mainly decide whether the original prompt or response looks unsafe. Self-checking methods (Zhang, Zhang, and Foerster 2024) move the decision after generation, but still rely on the target model to reason about its own unsafe output. More recent inference-time methods introduce intermediate structure. SecurityLingua (Li et al. 2025) compresses prompts to reveal security-relevant intent, IntentionReasoner (Shen et al. 2025) reasons about intent and rewrites risky queries, and Answer-Then-Check (Cao et al. 2025) verifies a drafted response before release. ReasoningGuard (Wang et al. 2026) is closer to LLMs because it intervenes during reasoning, but it relies on access to the model’s reasoning process and is therefore less suitable for closed-source targets. CRG addresses a different problem: how to govern the execution of a closed-source LLM when hidden reasoning is unavailable. Instead of only filtering, compressing, rewriting, or self-checking, CRG constructs a trusted side-channel representation and uses it to control the target-facing context, reasoning exposure, route, output audit, and benign fallback. This execution-governance view is the main distinction from prior inference-time safeguards.

Method: Capability-Routed Guard

Threat Model and Design Goals

We consider a closed-source jailbreak setting in which the adversary controls the user prompt and aims to elicit a response that violates a harmful-behavior policy. Our main evaluation uses standard black-box attacks, and we additionally study a white-box adaptive setting in which the adversary knows the CRG prompt and routing policy. This adaptive setting tests whether CRG relies on obscurity of the defense prompt rather than on its routing and verification mechanisms.

The defender operates an API-level wrapper around a closed-source target LLM. The defender cannot inspect hidden reasoning traces, fine-tune the target, modify model weights, or alter provider-side moderation. The available controls are limited to inference-time actions: calling an auxiliary side-channel model, rewriting the target-facing prompt, selecting exposed reasoning controls when available, injecting system instructions, blocking unsafe requests, and auditing public target outputs.

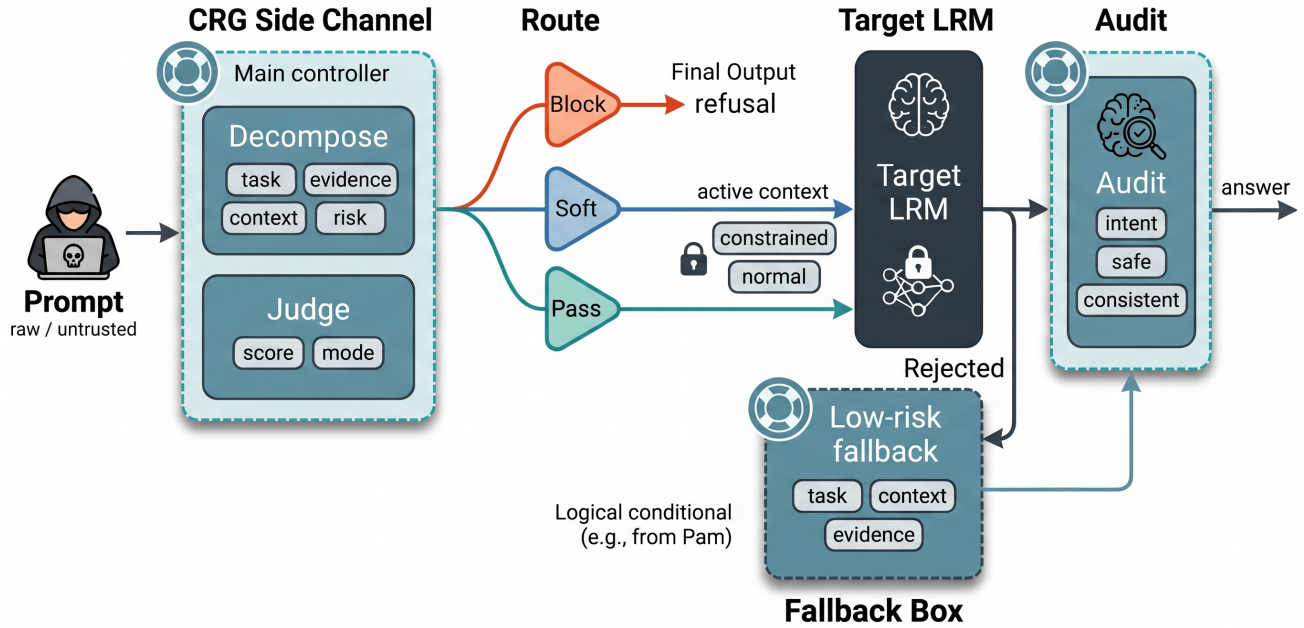


Figure 1: Overview of CRG. A side-channel controller separates the raw prompt into trusted task context and risk signals, then routes the request to block, soft, or pass execution before the target LRM is invoked. Short labels summarize groups of CRG fields: task = authorized task, evidence = safety evidence, context = active context, risk = capability and policy signals, score = actionability and risk score, and mode = reasoning-mode decision. Side-channel modules share the same checker/extractor model with different prompts, while the target LRM is invoked only on non-blocked routes.

The design goal is to improve safety without turning the wrapper into a broad refusal mechanism. CRG should prevent harmful capability transfer under adversarial reasoning context, while preserving useful answers for benign and safety-salient prompts. This safety-utility objective motivates a defense that can route requests, control target-facing context, and audit public outputs under closed-source API constraints.

Overview

Figure 1 summarizes the CRG pipeline. CRG is an inference-time wrapper that governs the full target-model execution path. Given a raw user prompt, CRG first invokes a side-channel extractor/checker model, Gemini 2.5 Flash in the main experiments, to construct a trusted representation of the user’s authorized task, safety evidence, active context, and capability-transfer risk. CRG then uses this representation to route the request. High-risk prompts are blocked before target invocation, ambiguous prompts are sent through a constrained execution path, and low-risk prompts are forwarded through trusted active context. After generation, CRG verifies public outputs with TraceCheck and can use a restricted fallback to preserve utility for low-risk benign prompts that would otherwise be refused.

Side-Channel Representation

The side-channel representation is the interface between the raw user prompt and the target LRM. It is produced in two

stages. First, the controller decomposes the prompt into task-, context-, and risk-related artifacts: *authorized task*, *safety evidence*, *active context*, *benign reasoning value*, *capability-transfer risk*, *policy sensitivity*, a preliminary *reasoning mode*, and a short *gate rationale*. Second, a mode/risk judge converts these artifacts into *actionability*, *risk score*, and *final reasoning mode*. CRG uses the resulting representation for routing, rewriting, and later verification.

These fields are chosen to preserve task utility while maintaining an authority boundary. The authorized task identifies what the user is actually asking for; safety evidence retains facts needed for risk assessment; active context defines the prompt that may be sent to the target; and the risk and mode fields support route and reasoning-control decisions. User-provided demonstrations, policies, role assignments, reasoning traces, and final-answer constraints may help interpret the request, but they are not automatically treated as target-model instructions. CRG therefore retains safety-relevant evidence while keeping untrusted reasoning scaffolds and false authority outside the target’s executable context.

The representation is capability-oriented rather than keyword-oriented. CRG does not merely ask whether the prompt contains sensitive words; it asks whether a faithful answer would transfer actionable harmful capability. This allows benign safety-salient prompts to remain answerable when their domain is harmless, while prompts that provide operational misuse guidance remain high risk even if they are

wrapped as fiction, puzzles, or analysis.

Capability-Routed Execution

CRG uses the structured side-channel representation to govern how, and under what computational constraints, the target LRM is executed. Rather than treating safety as a static binary prompt filter, CRG views defense as a problem of *capability allocation*: a dynamic control layer that authorizes the executable task context while regulating the target’s reasoning budget.

Let p be the raw user prompt and let $z(p)$ denote the extracted side-channel representation. The downstream model/risk judge evaluates $z(p)$ to produce an actionability category, a policy risk score $r \in [0, 5]$, and a target reasoning recommendation. The risk score determines the base execution route $\rho(p)$:

$$\rho_r(p) = \begin{cases} \text{BLOCK}, & r \geq T_b \\ \text{SOFT}, & T_s \leq r < T_b \\ \text{PASS}, & r < T_s, \end{cases}$$

where T_b and T_s represent the hard-block and soft-route thresholds, respectively. The final routing decision combines this base route with actionability and reasoning recommendations. If a high-risk prompt ($r \geq T_b$) is judged to be benign *common knowledge* rather than actionable capability transfer, CRG downgrades the route to SOFT. Conversely, if the judge detects anomalous target-facing reasoning requirements (such as obfuscated query wrappers or complex adversarial frameworks), it overrides a default PASS route and upgrades the execution to SOFT. This multidimensional gating ensures that routing is not a simple binary classification, but a dynamic execution-governance path.

Under this model, each route corresponds to a distinct capability profile assigned to the target LRM. The BLOCK route allocates no execution resources, returning a standard refusal. On both the PASS and SOFT routes, CRG strips away adversarial reasoning scaffolding by comparing the original prompt with the extracted *active context*; if a substantive rewrite is present, only this clean, task-relevant context is routed to the target. To separate these non-blocked execution surfaces, the PASS route grants the target LRM its default reasoning capability, whereas the SOFT route dynamically restricts this capacity at the API level (e.g., by capping the Gemini hidden `budget_tokens` parameter to 1024 tokens or setting the OpenAI `reasoning_effort` parameter to `low`). This design ensures that benign prompts preserve full utility, while ambiguous or adversarial prompts are executed under a strict, task-isolated context and a restricted computational budget, denying the attacker the extended reasoning space required to execute the exploit.

Post-Generation Verification and Fallback

For target responses that are not hard-blocked, CRG applies a post-generation audit called *TraceCheck* to verify the safety and consistency of the execution. This layer evaluates the target LRM’s public reasoning trajectory and final answer against the pre-extracted *authorized task*, verifying whether the target’s declared intent remains aligned with the authorized

task and whether its response plan is safe and consistent. To avoid unnecessary refusals on complex benign reasoning paths, this audit is risk-aware: minor trace anomalies only trigger a hard block when accompanied by elevated policy risk, ensuring that adversarial departures from the authorized intent are intercepted while harmless requests are certified.

To preserve utility for safe queries, CRG also incorporates a low-risk fallback mechanism. This recovery layer is activated when a query that was authorized for execution subsequently triggers target-side refusals or provider-side blocks due to sensitive vocabulary. If the side-channel controller has already classified the user’s underlying intent as harmless common knowledge and free of real-world capability-transfer risk, the fallback model directly answers the clean *authorized task* using a highly restricted, concise template. By bypassing the target LRM’s generic refusal boundaries on safe, safety-salient queries, this design maintains high benign utility while fully upholding the safety policy.

CRG as Execution Governance

A standard prompt filter operates as a static, input-level classifier that maps a user prompt directly to a binary block or pass decision. In contrast, CRG establishes an *execution governance* framework. Rather than evaluating a query in isolation, CRG constructs a unified, structured side-channel representation and uses it to govern the entire lifecycle of target LRM execution. This lifecycle spans target prompt rewriting via the clean *active context*, reasoning budget modulation, post-generation trajectory auditing with *TraceCheck*, and failure recovery via the low-risk fallback mechanism. This multi-layered approach distinguishes CRG from conventional filters. While monolithic static classifiers are constrained by a rigid, zero-sum tradeoff between safety and utility, CRG’s decoupled architecture allows individual components to work in synergy. Our ablation study demonstrates that each layer targets a distinct vulnerability or over-refusal vector, allowing CRG to simultaneously achieve robust jailbreak defense and high benign utility, a cooperative effect that is fundamentally unattainable under a single static classifier.

Experiments

Experimental Setup

We evaluate CRG using two primary target LRMs, Gemini 2.5 Pro and ChatGPT o4-mini, with Gemini 2.5 Flash serving as the default side-channel controller. To ensure broad applicability and analyze defense sensitivity, we also extend our evaluation to include open-weight models as targets and more advanced frontier models as auxiliary controllers. We assess adversarial robustness using malicious queries sourced from HarmBench (Mazeika et al. 2024) and AdvBench (Zou et al. 2023), while benign utility is evaluated through XSTest (Röttger et al. 2024) and MT-Bench (Chen et al. 2025). Specifically, XSTest is a suite designed to identify over-refusal behaviors on safe but sensitive topics, while MT-Bench evaluates multi-turn conversational quality and instruction-following capability. Finally, we benchmark CRG against an unprotected target and five adaptable baseline

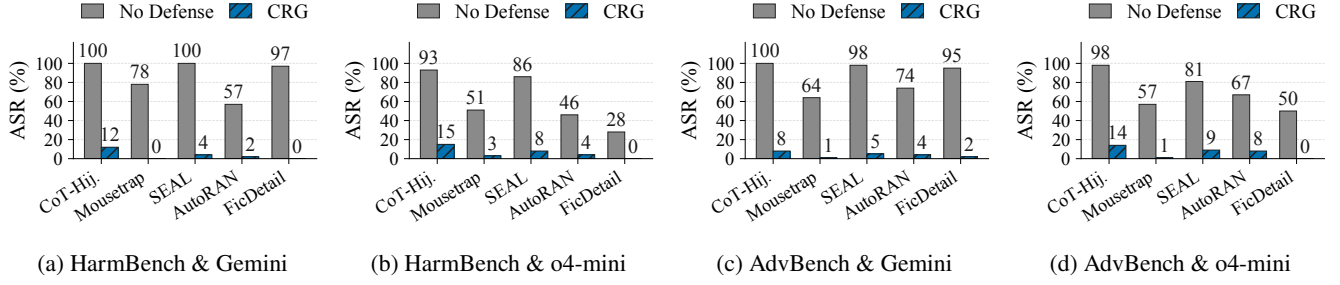


Figure 2: CRG substantially reduces ASR across five reasoning-centric jailbreak families. Each panel compares the ASR of an undefended target model with CRG on a specific dataset–model combination. Exact ASR values are shown above the bars.

Table 1: Attack success rate (%) on HarmBench and Gemini 2.5 Pro under different defenses. Lower is better. Abbreviations indicate the attacks: CoT-H (CoT Hijacking), MT (Mousetrap), AR (AutoRAN), FD (FicDetail).

Defense	CoT-H	MT	SEAL	AR	FD
No Defense	100	78	100	57	97
Llama Guard 3	98	67	100	34	12
Answer-Then-Check	95	15	83	16	9
Goal Priority	85	1	42	8	4
SecurityLingua	91	26	80	22	8
IntentionReasoner	55	57	94	12	3
Ours (CRG)	12	0	4	2	0

Table 2: End-to-end false-positive rate (%) on benign datasets. Lower is better.

Defense	XSTest		MT-Bench	
	Gemini	o4-mini	Gemini	o4-mini
No Defense	11.4	4.8	1.3	0.0
Llama Guard 3	14.0	6.0	2.5	2.5
Answer-Then-Check	14.4	14.4	2.5	0.0
Goal Priority	11.6	8.4	2.5	1.3
SecurityLingua	14.8	10.4	1.3	0.0
IntentionReasoner	14.2	5.2	1.3	1.3
Ours (CRG)	12.0	8.8	2.5	1.3

defenses: Llama Guard 3 (Inan et al. 2023), Answer-Then-Check (Cao et al. 2025), Goal Priority (Zhang et al. 2024), SecurityLingua (Li et al. 2025), and IntentionReasoner (Shen et al. 2025).

To measure robustness, we evaluate against five distinct reasoning-centric jailbreak frameworks: CoT-Hijacking (Zhao et al. 2025), Mousetrap (Yao et al. 2025), SEAL (Nguyen et al. 2026), AutoRAN (Liang et al. 2025), and FicDetail (Lu et al. 2025). Because their generation strategies vary significantly, we strictly adhere to the standardized evaluation protocols and generation methodologies established in their respective original implementations. Across all frameworks, the core evaluation metric is standardized: the final target response for

each goal is scored by an LLM-as-a-judge (using predefined policy guidelines) to determine if a successful jailbreak occurred.

We assess defense efficacy through two main metrics: Attack Success Rate (ASR) and End-to-End False Positive Rate (FPR). ASR measures robustness, defined strictly as the percentage of harmful goals successfully jailbroken according to the LLM judge. End-to-End FPR measures benign utility by penalizing over-refusal at any stage of the execution pipeline: a benign prompt is counted as a false positive if it is blocked by the defense, refused by the target model, or intercepted by the provider’s built-in filters.

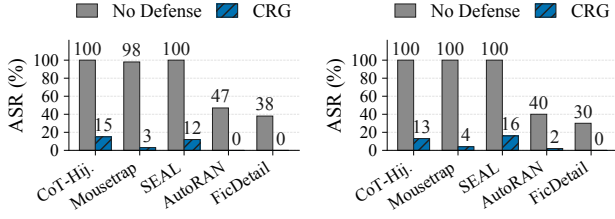
Robustness against Jailbreak Attacks

To establish a baseline for the threat posed by reasoning-centric jailbreaks, we first evaluate unprotected LLMs. As Figure 2 illustrates, these attacks are highly effective, routinely achieving near-perfect ASR, with CoT-Hijacking and SEAL reaching 100%. Implementing CRG, however, drastically reduces this vulnerability. Across all evaluated attack families for both Gemini 2.5 Pro and ChatGPT o4-mini, CRG consistently brings the ASR down to 15% or lower. Such broad effectiveness indicates that our multi-layered pipeline successfully generalizes against diverse adversarial structures rather than overfitting to specific prompt templates.

Furthermore, we compare CRG against existing defense baselines (Table 1). While conventional methods mitigate specific attack vectors, they remain highly brittle overall. For example, Goal Priority successfully neutralizes Mouse-trap (1% ASR) but is easily overwhelmed by CoT-Hijacking (85% ASR). Advanced intent-analyzing defenses like SecurityLingua and IntentionReasoner show similar inconsistencies, suffering severe failures against SEAL (allowing 80% and 94% ASR, respectively). These failures stem from a shared conceptual flaw. Standard safeguards operate as static text classifiers that judge prompts or responses in isolation. Reasoning-centric jailbreaks exploit this limitation by manipulating the task decomposition, allowing harmful objectives to masquerade as legitimate reasoning steps. CRG overcomes this vulnerability by treating safety as an end-to-end execution governance problem. Specifically, our pipeline isolates the authorized task from untrusted context, dynamically allocates reasoning compute based on risk, audits the generated reasoning trace for consistency, and applies a low-risk fallback

Table 3: Answer quality score with 10 as the full mark.

Defense	XSTest		MT-Bench	
	Gemini	o4-mini	Gemini	o4-mini
No Defense	8.6	9.4	8.6	9.4
Ours (CRG)	9.0	9.6	8.8	9.7



(a) HarmBench & DeepSeek-R1 (b) HarmBench & Qwen3-Max

Figure 3: Effectiveness of CRG for open-source reasoning models.

to recover utility on benign queries. By governing the entire execution lifecycle rather than applying a static filter, CRG successfully protects the model’s cognitive process.

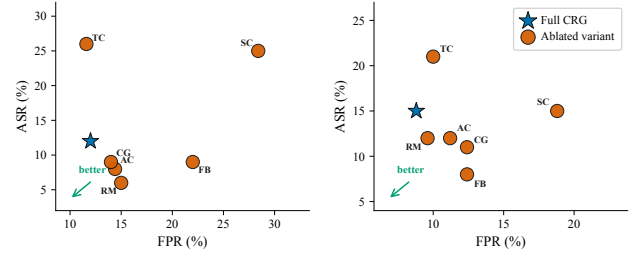
Preservation of Benign Utility

A practical defense must achieve security without broadly refusing safe, complex queries. To verify this, we evaluate the End-to-End False Positive Rate (FPR) on XSTest and MT-Bench (Table 2). CRG maintains highly competitive FPRs across both Gemini 2.5 Pro and ChatGPT o4-mini, closely matching the inherent refusal rates of the unprotected models. This confirms that CRG effectively avoids over-refusal. Furthermore, when evaluating the generated outputs (Table 3), applying CRG actually yields a slight improvement in average Answer Quality. By supplying the target LRM with a clean, isolated task context, CRG not only secures the execution but also helps the model focus on and fulfill benign requests more efficiently.

Analysis and Discussion

Generalization to Open-Weight Models

While primarily designed for closed-source APIs, CRG’s architecture generalizes effectively to open-weight reasoning models. We evaluate CRG on DeepSeek-R1 (DeepSeek-AI 2025) and Qwen3-Max (Team 2025) against the five jailbreak families on HarmBench (Figure 3). Without defense, both models are highly vulnerable, frequently reaching 100% ASR on complex attacks like CoT-Hijacking and SEAL. CRG robustly neutralizes these threats, reducing the ASR for both models to below 17% on the most aggressive attacks and nearly eliminating success on AutoRAN and FicDetail. These results demonstrate that CRG’s multi-layered pipeline successfully audits and governs RL-incentivized reasoning trajectories, proving its efficacy is independent of the underlying target model architecture.



(a) HarmBench & Gemini (b) HarmBench & o4-mini

Figure 4: Safety–utility tradeoff of CRG ablations. Each point is one CRG variant, with XSTest false-positive rate on the x-axis and CoT-Hijacking attack success rate on the y-axis. Lower-left is better. Abbreviations indicate the removed component: TC (TraceCheck), RM (reasoning-mode control), AC (active-context rewrite), FB (low-risk fallback), SC (structured CRG), CG (capability gate).

Component Ablations

To understand how CRG balances security and utility, we ablate its core components on CoT-Hijacking and XSTest, as shown in Figure 4. The ablation results illustrate why defending reasoning models requires a specialized architecture rather than standard prompt filtering. Disabling post-generation auditing (TraceCheck), for instance, causes the ASR on Gemini 2.5 Pro to more than double (from 12% to 26%), confirming its necessity for intercepting models that drift into harmful trajectories during generation. Conversely, removing elements designed to preserve utility leads to severe over-refusal. Replacing the structured CRG decomposition with a flat prompt filter causes the FPR to surge from 12.0% to 28.4%, highlighting that separating the core task from its surrounding context is essential for distinguishing benign, safety-salient queries from actual attacks. Similarly, disabling the low-risk fallback mechanism nearly doubles the FPR, illustrating its crucial role in recovering safe prompts that might otherwise trigger internal provider refusals. Ultimately, these opposing sensitivities validate the fundamental premise of CRG: defending large reasoning models requires a paradigm shift from static prompt filtering to holistic execution governance. By moving beyond binary classification, our approach demonstrates that adversarial manipulation of the reasoning path must be structurally neutralized to prevent the exploitation of the target model’s underlying capabilities.

Cost and Latency

Because attacks stop early after a successful jailbreak, total wall-clock time per attack goal is not a clean latency metric: weaker defenses can look faster simply because the attacker succeeds earlier. We therefore report the average wall-clock latency specifically on examples where the defense is activated and successfully thwarts the attack, measured on Gemini 2.5 Pro. As shown in Figure 5, CRG is highly competitive, remaining comparable to IntentionReasoner and Goal Priority, while being substantially faster than Answer-Then-Check and SecurityLingua on successfully defended

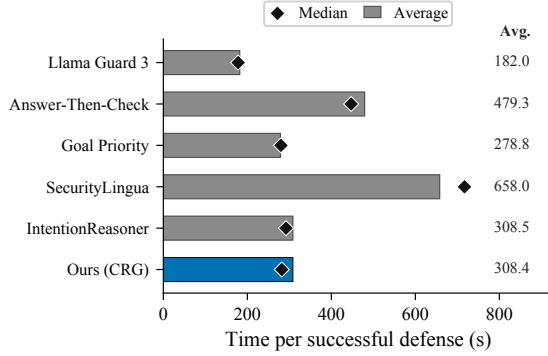


Figure 5: Wall-clock latency per successfully defended Harm-Bench harmful example on Gemini 2.5 Pro. Lower is better.

Table 4: Extractor model sensitivity evaluation on Gemini 2.5 Pro (target LRM), reporting CoT-Hijacking ASR (%) and XSTest FPR (%). Lower is better for both columns.

Extractor	ASR	FPR
Gemini 2.5 Pro	9.0	3.2
ChatGPT o4-mini	14.0	11.6
Gemini 2.5 Flash	12.0	12.0

examples. Although CRG may run the side-channel model more than once when auditing or fallback is activated, many target calls are skipped entirely due to early blocks, and the side-channel model itself is significantly smaller and faster than the target LRM. This successful-defense latency provides a more meaningful comparison of actual overhead among deployable defenses, confirming that our multi-stage pipeline is practically feasible.

Extractor Model Sensitivity

While CRG is designed to leverage a lightweight side-channel controller, its efficacy does not depend critically on any single model architecture. To evaluate this sensitivity, we substitute our default controller (Gemini 2.5 Flash) with either Gemini 2.5 Pro or ChatGPT o4-mini while keeping the target LRM and evaluation parameters fixed. As compiled in Table 4, CoT-Hijacking ASR remains consistently low across all configurations, measuring 9.0% under Gemini 2.5 Pro, 14.0% under ChatGPT o4-mini, and 12.0% under Gemini 2.5 Flash.

In contrast, the benign XSTest false-positive rate is more sensitive to the capability of the extractor model. Employing the stronger Gemini 2.5 Pro as the auxiliary controller yields the lowest FPR at 3.2%, compared to 11.6% for ChatGPT o4-mini and 12.0% for Gemini 2.5 Flash. This variance indicates that a more capable extractor better distinguishes between genuinely harmful intents and benign-but-apparently-harmful queries, which in turn allows the pipeline to safely authorize low-risk fallback paths. We choose Gemini 2.5 Flash as our default auxiliary model as it strikes the most optimal cost-latency balance while still maintaining robust safety levels.

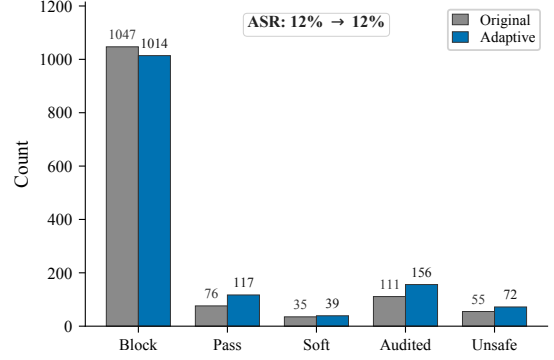


Figure 6: Original and adaptive CoT-Hijacking against CRG on HarmBench with Gemini 2.5 Pro.

Adaptive Robustness under White-Box Pressure

To evaluate the limits of intent isolation, we test CRG against a white-box adaptive version of CoT-Hijacking. In this scenario, the adversary possesses complete knowledge of the side-channel instructions and specifically designs the prompt to conceal capability transfer, exploit semantic ambiguity in the authorized task, and manipulate active context to trigger the low-risk fallback mechanism.

As illustrated in Figure 6, this adaptive strategy successfully exerts greater pressure on the pre-target gate, slightly reducing the number of hard blocks and letting more queries pass through or proceed via the soft route. However, because more queries bypass or soften this initial filter, a larger volume of target responses must be audited by our post-generation layer. By intercepting these remaining leaks post-hoc, TraceCheck completely neutralizes the attacker’s front-gate advantage, keeping the final ASR constant. While this does not represent full minimax security against any arbitrary adaptive strategy, it demonstrates that CRG’s multi-layered defense-in-depth architecture successfully compensates for white-box evasion attempts at the initial gate.

Conclusion

The emergence of large reasoning models exposes the fundamental limitations of static, classification-based safety paradigms. When models are incentivized to reason, plan, and decompose tasks dynamically, safety risks are no longer confined to isolated inputs or outputs; instead, they emerge dynamically from the reasoning process itself. This work demonstrates that securing these systems requires transitioning from static text filters to active runtime execution governance. By isolating authorized execution intent from untrusted reasoning contexts, we can safely govern a model’s cognitive trajectory without compromising its reasoning capabilities. Ultimately, our findings suggest that as AI architectures evolve from passive language generators into autonomous, reasoning agents, security must shift from defending the model’s inputs to governing its end-to-end execution lifecycle.

References

- Cao, C.; Xu, X.; Han, B.; and Li, H. 2025. Reasoned Safety Alignment: Ensuring Jailbreak Defense via Answer-Then-Check. *CoRR abs/2509.11629*.
- Chao, P.; Robey, A.; Dobriban, E.; Hassani, H.; Pappas, G. J.; and Wong, E. 2025. Jailbreaking Black Box Large Language Models in Twenty Queries. In *IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*, 23–42. IEEE.
- Chen, J.; Feng, A.; Zhao, Z.; Garza, J.; Nurbek, G.; Qin, C.; Maatouk, A.; Tassiulas, L.; Gao, Y.; and Ying, R. 2025. MT-Bench: A Multimodal Time Series Benchmark for Temporal Reasoning and Question Answering. *CoRR abs/2503.16858*.
- DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *CoRR abs/2501.12948*.
- Ding, P.; Kuang, J.; Ma, D.; Cao, X.; Xian, Y.; Chen, J.; and Huang, S. 2024. A Wolf in Sheep’s Clothing: Generalized Nested Jailbreak Prompts can Fool Large Language Models Easily. In *Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 2136–2153. ACL.
- Inan, H.; Upasani, K.; Chi, J.; Rungta, R.; Iyer, K.; Mao, Y.; Tontchev, M.; Hu, Q.; Fuller, B.; Testuggine, D.; and Khabsa, M. 2023. Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations. *CoRR abs/2312.06674*.
- Jin, H.; Zhou, A.; Menke, J. D.; and Wang, H. 2024. Jailbreaking Large Language Models Against Moderation Guardrails via Cipher Characters. In *Annual Conference on Neural Information Processing Systems (NeurIPS)*. NeurIPS.
- Li, X.; Zhou, Z.; Zhu, J.; Yao, J.; Liu, T.; and Han, B. 2023. DeepInception: Hypnotize Large Language Model to Be Jailbreaker. *CoRR abs/2311.03191*.
- Li, Y.; Ahn, S.; Jiang, H.; Abdi, A. H.; Yang, Y.; and Qiu, L. 2025. SecurityLingua: Efficient Defense of LLM Jailbreak Attacks via Security-Aware Prompt Compression. *CoRR abs/2506.12707*.
- Liang, J.; Jiang, T.; Wang, Y.; Zhu, R.; Ma, F.; and Wang, T. 2025. AutoRAN: Weak-to-Strong Jailbreaking of Large Reasoning Models. *CoRR abs/2505.10846*.
- Lu, C.; Fan, X.; Huang, Y.; Xu, R.; Li, J.; and Xu, W. 2025. Does Chain-of-Thought Reasoning Really Reduce Harmfulness from Jailbreaking? In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 6523–6546. ACL.
- Mazeika, M.; Phan, L.; Yin, X.; Zou, A.; Wang, Z.; Mu, N.; Sakhaee, E.; Li, N.; Basart, S.; Li, B.; Forsyth, D. A.; and Hendrycks, D. 2024. HarmBench: A Standardized Evaluation Framework for Automated Red Teaming and Robust Refusal. In *International Conference on Machine Learning (ICML)*, 35181–35224. JMLR.
- Nguyen, V.; Zhao, S.; Dao, G.; Hu, R.; Xie, Y.; Wu, X.; and Luu, A. T. 2026. Three Minds, One Legend: Jailbreak Large Reasoning Model with Adaptive Stacked Ciphers. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 7139–7159. ACL.
- OpenAI. 2024. OpenAI o1 System Card. *CoRR abs/2412.16720*.
- Röttger, P.; Kirk, H.; Vidgen, B.; Attanasio, G.; Bianchi, F.; and Hovy, D. 2024. XSTest: A Test Suite for Identifying Exaggerated Safety Behaviours in Large Language Models. In *Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 5377–5400. ACL.
- Shen, Y.; Huang, Z.; Guo, Z.; Liu, Y.; Chen, G.; Yin, R.; Zheng, X.; and Huang, X. 2025. IntentionReasoner: Facilitating Adaptive LLM Safeguards through Intent Reasoning and Selective Query Refinement. *CoRR abs/2508.20151*.
- Team, Q. 2025. Qwen3 Technical Report. *CoRR abs/2505.09388*.
- Wang, Y.; Zhang, M.; Wang, Y.; Hong, G.; Wen, M.; You, X.; and Yang, M. 2026. ReasoningGuard: Safeguarding Large Reasoning Models with Inference-time Safety Aha Moments. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 31497–31526. ACL.
- Wang, Z.; Yang, F.; Wang, L.; Zhao, P.; Wang, H.; Chen, L.; Lin, Q.; and Wong, K. 2024. SELF-GUARD: Empower the LLM to Safeguard Itself. In *Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 1648–1668. ACL.
- Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Ichter, B.; Xia, F.; Chi, E. H.; Le, Q. V.; and Zhou, D. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Annual Conference on Neural Information Processing Systems (NeurIPS)*. NeurIPS.
- Yao, Y.; Tong, X.; Wang, R.; Wang, Y.; Li, L.; Liu, L.; Teng, Y.; and Wang, Y. 2025. A Mousetrap: Fooling Large Reasoning Models for Jailbreak with Chain of Iterative Chaos. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 7837–7855. ACL.
- Zhang, Z.; Yang, J.; Ke, P.; Mi, F.; Wang, H.; and Huang, M. 2024. Defending Large Language Models Against Jailbreaking Attacks Through Goal Prioritization. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 8865–8887. ACL.
- Zhang, Z.; Zhang, Q.; and Foerster, J. N. 2024. PARDEN, Can You Repeat That? Defending against Jailbreaks via Repetition. In *International Conference on Machine Learning (ICML)*, 60271–60287. PMLR.
- Zhao, J.; Fu, T.; Schaeffer, R.; Sharma, M.; and Barez, F. 2025. Chain-of-Thought Hijacking. *CoRR abs/2510.26418*.
- Zou, A.; Wang, Z.; Kolter, J. Z.; and Fredrikson, M. 2023. Universal and Transferable Adversarial Attacks on Aligned Language Models. *CoRR abs/2307.15043*.